EVENTEASE

Development of a RESTful Web API for an Event Ticketing and Venue Booking System

SWC3633/SWC4443 — Web API Development

Student name: AMZAR FAIZZUDDIN BIN MOHD HAMDAN
Student ID: AM2508020090
GitHub repository: [ADD URL]
Demonstration video: [ADD ACCESSIBLE LINK]
Submission date: 20 October 2026

1. System Overview

EventEase is a web application prototype for discovering events and managing ticket bookings. Its intended roles are Administrator, Organiser and Customer. Customers can browse published events, register/login, create and cancel bookings, and inspect booking status. Organisers manage events they own. Administrators have wider system-management privileges. The frontend uses HTML, CSS and JavaScript Fetch; the backend is a PHP JSON REST API backed by MySQL.

2. Database Design

The relational schema includes users, venues, events, bookings, payments and api_tokens. Primary keys identify rows; foreign keys connect organisers to events, venues to events, customers to bookings, events to bookings, bookings to payments and users to tokens.

Entity Relationship Diagram (ERD)

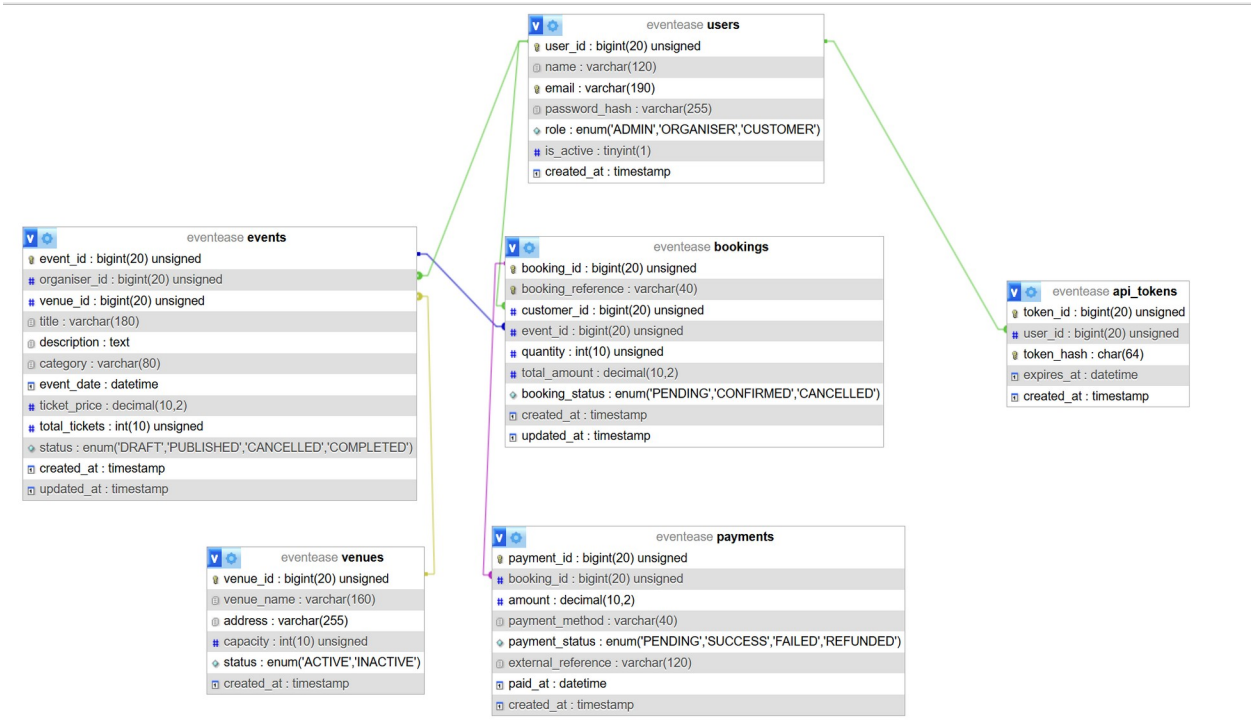


Figure 1: Entity Relationship Diagram (ERD) of the EventEase Database

3. RESTful API Documentation

All endpoints are served under the base URL http://localhost/EventEase/backend/api/v1 and exchange JSON. Protected endpoints require the header Authorization: Bearer <JWT> obtained from POST /auth/login. The administrator metrics endpoint additionally requires the X-API-Key header.

Every response uses a consistent envelope: successful responses return {"success": true, "data": ...} and failed responses return {"success": false, "error": {"code": ..., "message": ...}}.

3.1 Endpoint summary

Method	URL	Access	Request parameters / body	Success	Main error codes
POST	/auth/register	Public	name, email, password (min. 10 chars)	201 user_id	400 INVALID_JSON, 422 INVALID_EMAIL / WEAK_PASSWORD, 409 EMAIL_EXISTS
POST	/auth/login	Public	email, password	200 access_token, expires_at, user	401 LOGIN_FAILED, 422, 429 RATE_LIMITED
GET	/auth/me	Any signed-in user	—	200 current user	401
POST	/auth/logout	Any signed-in user	—	200 token revoked	401
GET	/events	Public (manage=1: Admin, Organiser)	page, limit (max 50), q, category, sort (event_date | title | ticket_price), direction, manage	200 data + meta {page, limit}	401, 403
GET	/events/{id}	Public for PUBLISHED events	—	200 event incl. tickets_available	404
POST	/events	Admin, Organiser	venue_id, title, category, event_date, ticket_price, total_tickets, description, status	201 event_id	422 INVALID_EVENT_DATA / INVALID_EVENT_DATE / INVALID_VENUE / CAPACITY_EXCEEDED, 403
PUT / PATCH	/events/{id}	Admin, owning Organiser	all event fields incl. status	200	403, 404, 422
DELETE	/events/{id}	Admin	—	204	404, 409 RESOURCE_IN_USE
GET	/venues, /venues/{id}	Public	—	200	404
POST	/venues	Admin	venue_name, address, capacity, status	201 venue_id	422 INVALID_CAPACITY / INVALID_VENUE_STATUS
PUT / PATCH	/venues/{id}	Admin	venue_name, address, capacity, status	200	404, 422
DELETE	/venues/{id}	Admin	—	204	404, 409 RESOURCE_IN_USE
GET	/users, /users/{id}	Admin	—	200 (no password hashes)	401, 403, 404
POST	/users	Admin	name, email, password, role	201 user_id	422 INVALID_USER_DATA

PUT / PATCH	/users/{id}	Admin	name, email, role, is_active	200	404, 422
DELETE	/users/{id}	Admin	—	204	409 SELF_DELETE_BLOCKED / RESOURCE_IN_USE
GET	/bookings, /bookings/{id}	Customer (own), Organiser (own events), Admin (all)	—	200 bookings with payment status	401, 404
POST	/bookings	Customer	event_id, quantity (1–10)	201 booking_reference, PENDING	403, 404, 409 INSUFFICIENT_TICKETS, 422 INVALID_QUANTITY
PATCH	/bookings/{id}/cancel	Customer (own), Organiser, Admin	—	200 CANCELLED	404, 409 ALREADY_CANCELLED / CANCELLATION_CLOSED
PATCH	/bookings/{id}/demo-confirmation	Admin	—	200 CONFIRMED (simulated payment)	409 BOOKING_CANCELLED
GET	/bookings/{id}/qr	Booking owner, Organiser, Admin	—	200 image_base64 (QR PNG)	409 TICKET_NOT_CONFIRMED, 502 QR_PROVIDER_ERROR
GET	/admin/metrics	Admin + X-API-Key	—	200 users, events, bookings counts	401 API_KEY_REQUIRED, 403

Design note: bookings are cancelled with PATCH /bookings/{id}/cancel instead of being deleted, so that booking and payment history is preserved. Cancelling a paid booking marks its payment as REFUNDED. The demo-confirmation endpoint simulates payment for coursework only; no real payment gateway is used.

3.2 Sample request and response

Login request (POST /auth/login):

```
{
  "email": "sofia@eventease.test",
  "password": "********"
}
```

Response format (200 OK):

```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "token_type": "Bearer",
    "access_token": "<JWT>",
    "expires_at": "<date-time>",
    "user": { "user_id": 4, "name": "Sofia Customer",
              "email": "sofia@eventease.test", "role": "CUSTOMER" }
```

```
  }
}
```

Booking request (POST /bookings, customer JWT):

```
{ "event_id": 1, "quantity": 1 }
```

Response format (201 Created):

```
{
  "success": true,
  "message": "Booking created and awaiting payment confirmation.",
  "data": { "booking_id": 12, "booking_reference": "EE-XXXXXXXXXX",
            "quantity": 1, "total_amount": 80, "booking_status": "PENDING" }
}
```

Validation error format (422 Unprocessable Entity):

```
{
  "success": false,
  "error": { "code": "INVALID_QUANTITY",
             "message": "Quantity must be an integer from 1 to 10." }
}
```

4. API Testing

The API was tested in Postman against the local XAMPP deployment using the exported collection EventEase API Tests (docs/EventEase.postman_collection.json). Collection variables hold the base URL and API key, and the login requests store the returned JWT in collection variables for later requests. The table below lists the tests whose results were captured; the screenshots are reproduced in Appendix A and Appendix B.

No.	Test case	Request	Expected	Recorded result	Evidence
1	Public event list with pagination and sorting	GET /events?page=1&limit=10&sort=event_date&direction=ASC	200 OK, JSON list	200 OK	Appendix A
2	Public search	GET /events?q=Workshop&sort=title&direction=ASC	200 OK, matching events	200 OK	Appendix A
3	Protected endpoint without JWT	GET /bookings (no Authorization header)	401 AUTH_REQUIRED	401 Unauthorized	Appendix A
4	Customer views own bookings	GET /bookings (customer JWT)	200 OK	200 OK	Appendix A
5	Invalid booking quantity	POST /bookings {"event_id":1,"quantity":0}	422 INVALID_QUANTITY	422 Unprocessable Entity	Appendix A, Section 9
6	Registration with invalid email	POST /auth/register (email "not-an-email")	422 INVALID_EMAIL	422 Unprocessable Entity	Appendix A, Section 9
7	Admin metrics with JWT and API key	GET /admin/metrics (admin JWT + X-API-Key)	200 OK with counts	200 OK	Appendix A, Appendix B
8	Invalid or expired JWT	Protected request with an invalid token	401	401 Unauthorized	Appendix B

9	Administrator lists all bookings	GET /bookings (admin JWT)	200 OK	200 OK, Postman assertion passed (1/1)	Appendix B

In addition, the demonstration video shows the frontend flow of browsing events, booking tickets, viewing booking status and displaying a QR ticket for a confirmed booking. Venue and user management endpoints are documented in Section 3 but are not yet covered by separate Postman requests.

5. System Implementation

The backend uses PDO prepared statements for database access and a JSON router. Authentication is implemented with HS256 JWTs, expiry claims and a server-side hashed-token record for revocation. Cross-cutting middleware helpers provide request logging and login rate limiting. Public event list requests support pagination, search, category filtering and allow-listed sorting. The frontend consumes the API with Fetch and displays user feedback.

6. System Demonstration

The following screenshots document the running EventEase frontend. They show the login page, home page, published event listing, customer bookings and organiser event management.

Figure 2: Login page

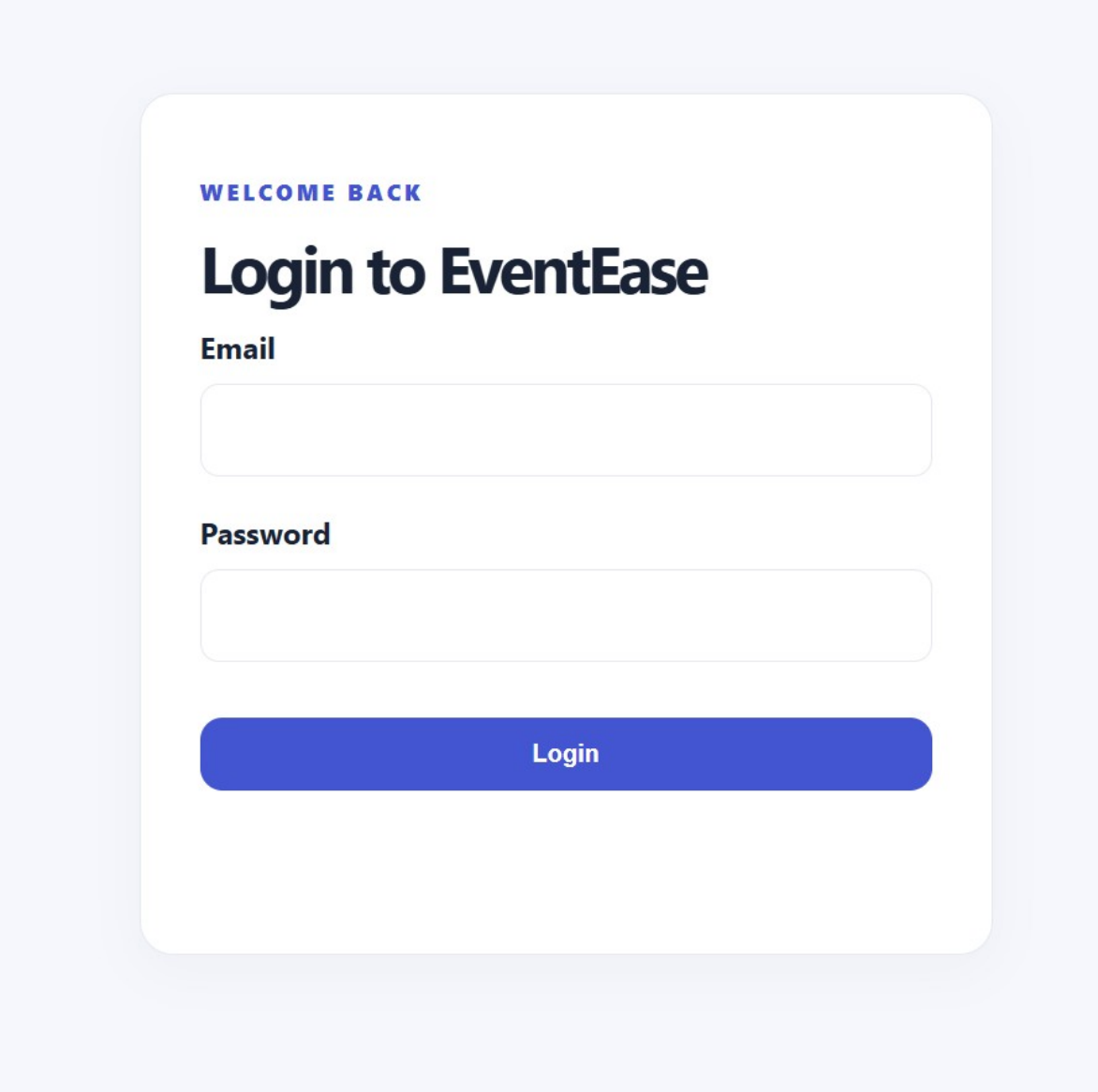


Figure 3: EventEase home page

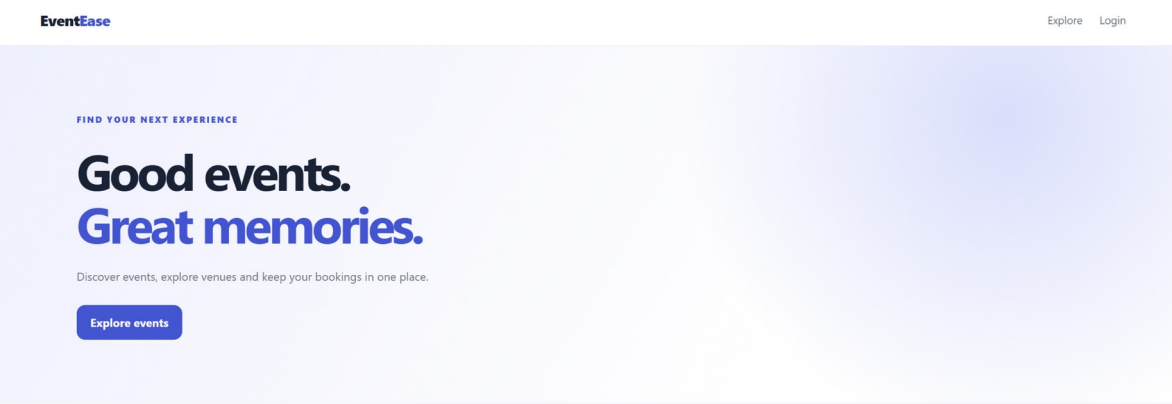


Figure 4: Explore Events page showing published events and ticket availability

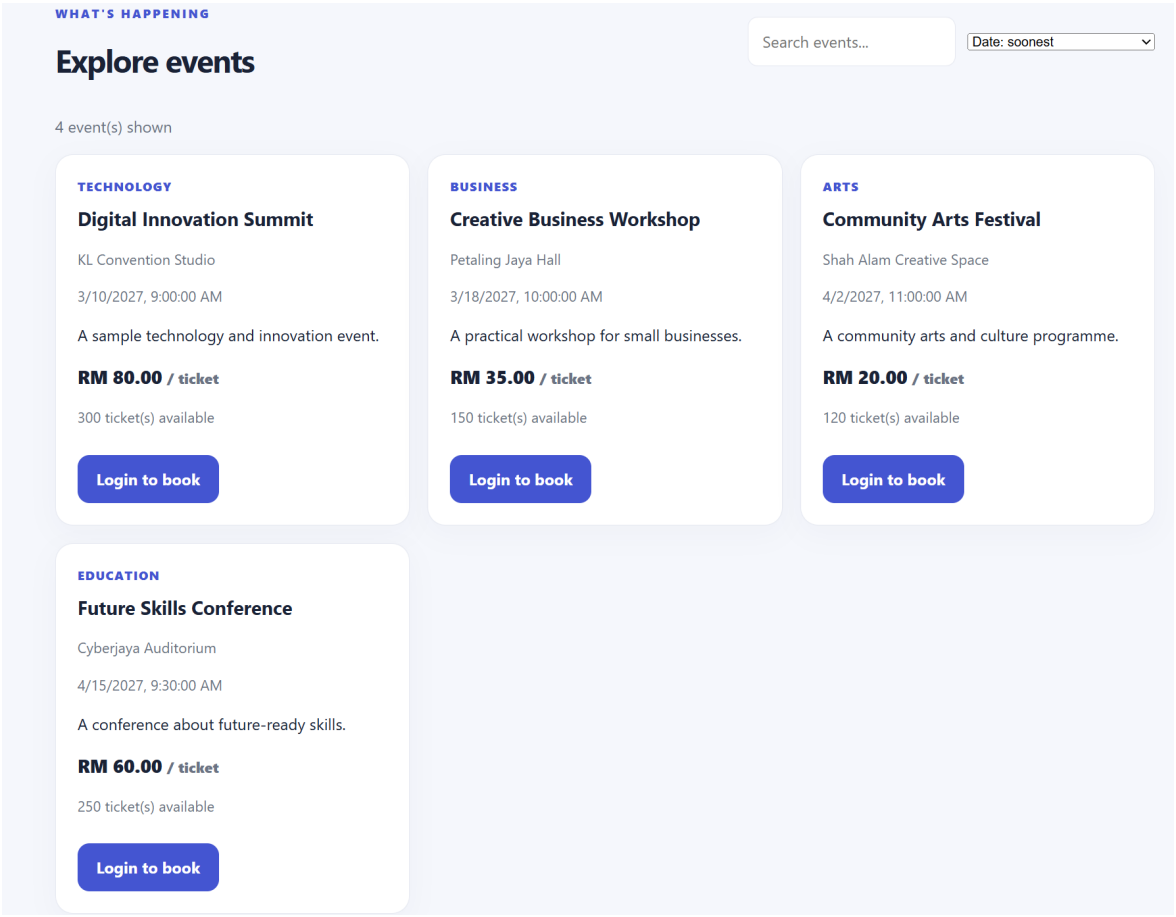


Figure 5: My Bookings page showing booking references, amounts and statuses

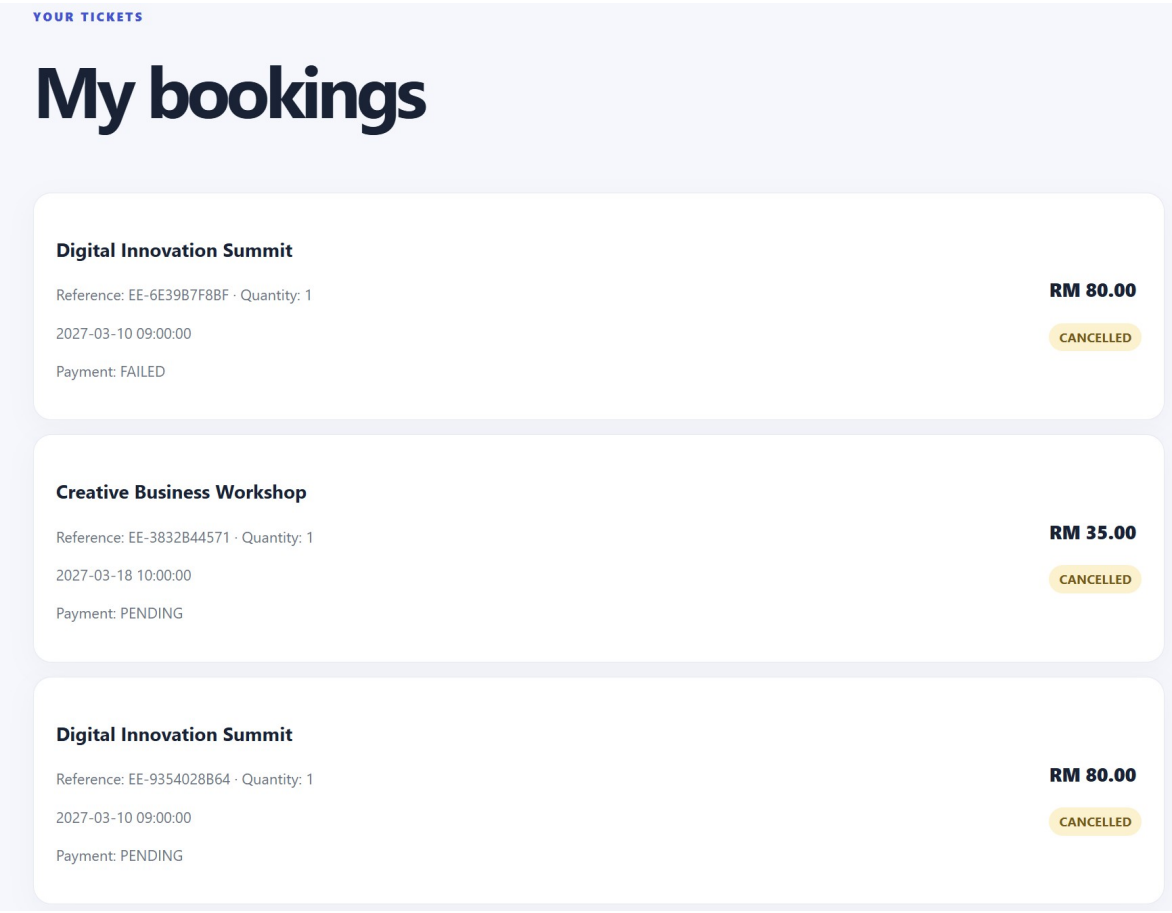


Figure 6: Navigation menu after login, including booking and event management options

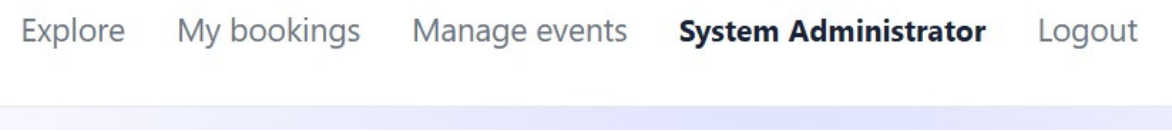


Figure 7: Organiser Manage Events page and Create Event form

ORGANISER WORKSPACE

Manage events

Create event

Event title

Description

Category

Venue

Cyberjaya Auditorium (capacity 400)

Date and time

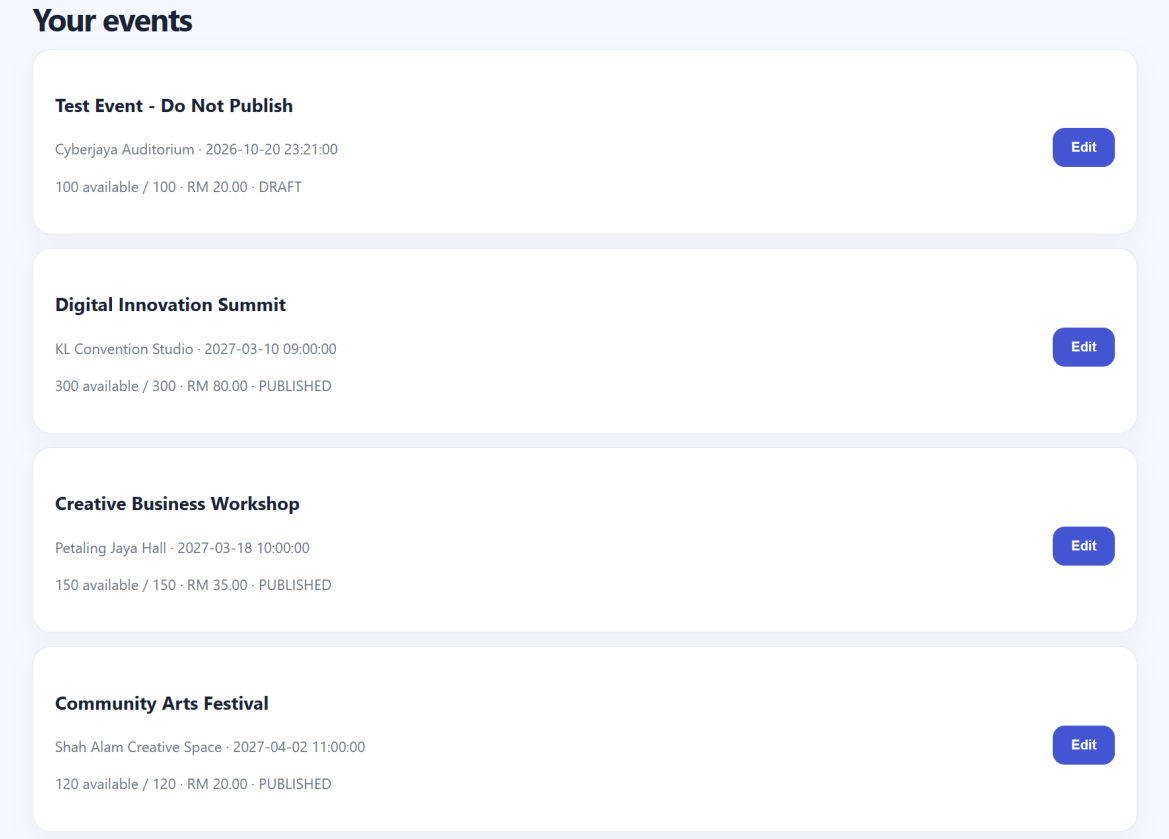
dd/mm/yyyy --:-- --

Ticket price (RM)

Total tickets

Status

Figure 8: Your Events list showing event status and available tickets



7. Project Repository

GitHub URL: [paste working repository URL]. The repository should include source code, SQL scripts, documentation and Postman collection. Exclude .env, logs, access tokens, passwords and generated local data.

8. Additional/Advanced Features

JWT authentication. Login issues an HS256-signed JWT containing the claims iss, sub, email, role, iat, exp and jti, valid for 8 hours. A SHA-256 hash of each token is stored in the api_tokens table. On every protected request the API verifies the signature and expiry, checks that the token hash still exists (so logout revokes the token), and confirms that the user is active and the role matches.

Role-based access control. Three roles are enforced on the server: Customers browse events and create, view and cancel their own bookings; Organisers create and edit only the events they own and see bookings for those events; Administrators manage users, venues and all events and bookings. Requests from the wrong role receive 403 FORBIDDEN, and records that belong to another user are reported as 404 so that their existence is not revealed.

API key protection. GET /admin/metrics requires both an administrator JWT and a valid X-API-Key header. The key is compared with hash_equals() and is loaded from the environment rather than written in the report.

Request validation and centralised error handling. Required fields, email format, password length, quantities, prices, dates, venue capacity and status values are validated before database writes, returning 422 with a specific error code. A single try/catch in the router converts database and unexpected errors into generic 409 or 500 JSON responses, so SQL messages are written to the server log instead of being returned to the client. Unknown routes return 404 and unsupported methods return 405.

Third-party API: goQR QRServer. GET /bookings/{id}/qr builds a JSON payload containing the booking reference, event title, event date and quantity, and sends it to https://api.qrserver.com/v1/create-qr-code/ using PHP cURL with a 4-second connection timeout and an 8-second total timeout. The QR code is only issued for CONFIRMED bookings (otherwise 409 TICKET_NOT_CONFIRMED). The PNG returned by QRServer is base64-encoded in the JSON response and displayed by the My Bookings page. If the provider fails, the API returns 502 QR_PROVIDER_ERROR.

Request logging. A shutdown handler writes one JSON line per request to backend/logs/api.log containing the time, HTTP method, path, status code, duration in milliseconds and client IP address. The log folder is excluded from the repository.

Login rate limiting. Each combination of client IP address and email is limited to 10 login attempts in a five-minute window; further attempts return 429 RATE_LIMITED.

Pagination, search, filtering and sorting. GET /events accepts page and limit (limit capped at 50), a keyword q that searches title, description and category, a category filter, and sort/direction parameters. Sort columns are taken from an allow-list so user input is never inserted into the ORDER BY clause.

Transactional booking. Booking creation runs inside a database transaction that locks the event row with SELECT … FOR UPDATE, recalculates booked tickets and rejects the request with 409 INSUFFICIENT_TICKETS if capacity would be exceeded, preventing overselling when two customers book at the same time.

9. Debugging and Performance Optimisation

The following issues were encountered during local setup and API testing. The fixes below describe the actions taken during this project; no quantitative performance benchmark was recorded.

Issue 1 — MySQL port conflict: The XAMPP database service could not start because the Windows MySQL80 service was using port 3306. The MySQL80 service was stopped, after which MariaDB in XAMPP could be started and the eventease database opened in phpMyAdmin.

Issue 2 — Authorization header forwarding: Protected API requests initially required the Bearer JWT to reach PHP correctly. The Apache .htaccess configuration was updated to preserve the Authorization header using HTTP_AUTHORIZATION, and protected requests could then be tested with a token.

Issue 3 — API input validation: Postman tests were used to verify that invalid inputs were rejected. A booking quantity outside the accepted range returned 422 Unprocessable Entity with INVALID_QUANTITY, and an invalid email address during registration returned 422 Unprocessable Entity with INVALID_EMAIL.

Issue 4 — Authentication and access control checks: A protected endpoint requested without a Bearer token returned 401 Unauthorized with AUTH_REQUIRED. Admin metrics returned 200 OK when tested with an administrator JWT and the required API key.

Performance optimisation note: No before-and-after load test or response-time benchmark was recorded, so this report does not claim a measured performance improvement. The implementation uses PDO prepared statements and pagination/search options as design measures; their performance impact was not separately benchmarked.

10. Individual Reflection and Contribution Record (200–300 words)
Draft—edit this to match your actual experience and approved solo arrangement:

I worked on the EventEase event-ticketing and venue-booking prototype using PHP, MySQL, HTML, CSS and JavaScript. My responsibilities included setting up the local XAMPP environment, importing the relational database, testing the API through the browser, and checking the user flow from browsing events to booking and viewing bookings. During setup, MySQL could not start because another MySQL service was occupying port 3306. I identified the running service and stopped it so XAMPP could use the port. I also encountered a routing issue where a direct event endpoint returned an Apache 404 page instead of JSON. I added rewrite configuration and retested the versioned API route. Another challenge involved authentication: the booking page redirected to login when the bearer token was not correctly accepted. I checked the frontend/API flow and verified that a booking could subsequently appear in My Bookings.

This process helped me understand that a REST API project is more than a working interface: it also requires authentication, role-based access, validation, correct status codes, database integrity and

repeatable testing. I learned to test one change at a time and preserve the database while debugging. My remaining improvements are to verify all endpoint behaviours in Postman, test role isolation and ticket-capacity rules, and capture reproducible evidence. The lecturer has permitted me to work individually; I will document that arrangement accurately and confirm how the group-video requirement should be handled.

Contribution evidence: [list only your actual files, commits, tests and screenshots].

Appendix A: Postman API Testing Evidence

The following screenshots show selected responses captured during local testing of the EventEase API. Status codes and response bodies are visible in each screenshot. Login token screenshots are intentionally excluded to avoid exposing bearer credentials.

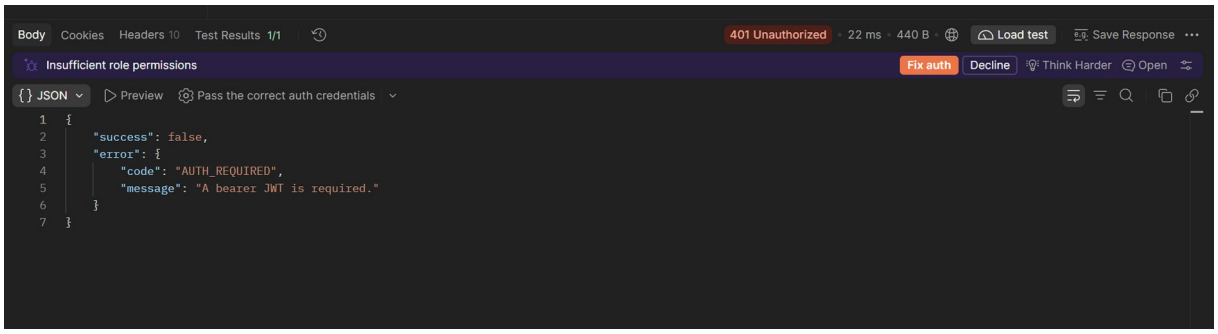
Public events list — 200 OK



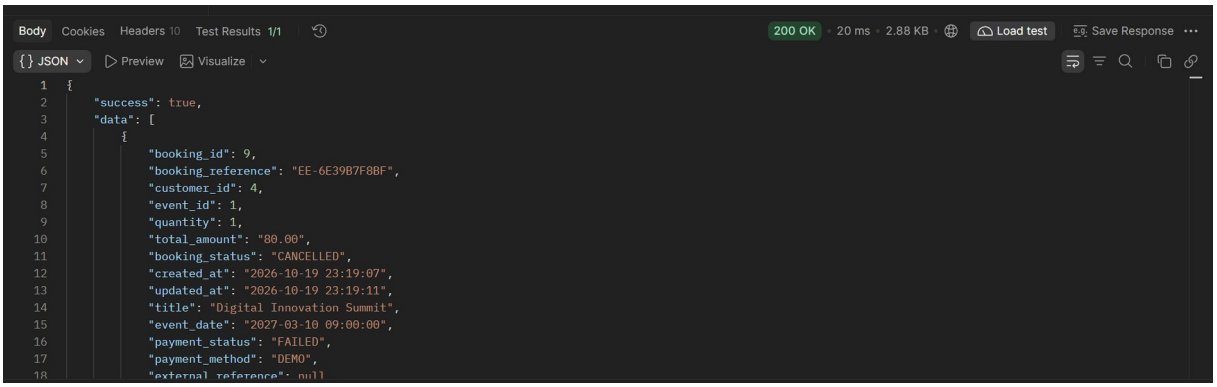
Public search/filter result — 200 OK



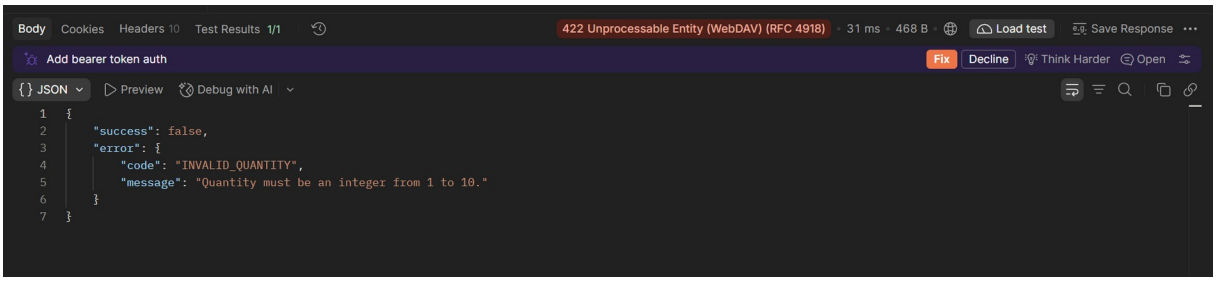
Protected endpoint without JWT — 401 Unauthorized



My bookings response — 200 OK



Invalid booking quantity — 422 Unprocessable Entity



Registration with invalid email — 422 Unprocessable Entity



Admin metrics with API key — 200 OK



Appendix B — Latest Postman Test Evidence (20 October 2026)

This appendix records the latest verification performed in Postman. Screenshots are included as supporting evidence. Tokens are not reproduced in this report.

Test	Observed result	Evidence / meaning
Admin login	200 OK	A fresh administrator JWT was issued successfully.
Admin metrics with valid API key	200 OK	Returned users: 5, events: 6, bookings: 11. The Postman test passed.
Admin metrics with incorrect API key	401 Unauthorized	The API rejected an incorrect X-API-Key.
Bookings with invalid/expired JWT	401 Unauthorized	The API rejected an invalid or expired JWT.
Bookings with fresh admin JWT	200 OK	The bookings endpoint returned booking records; Postman test passed 1/1.

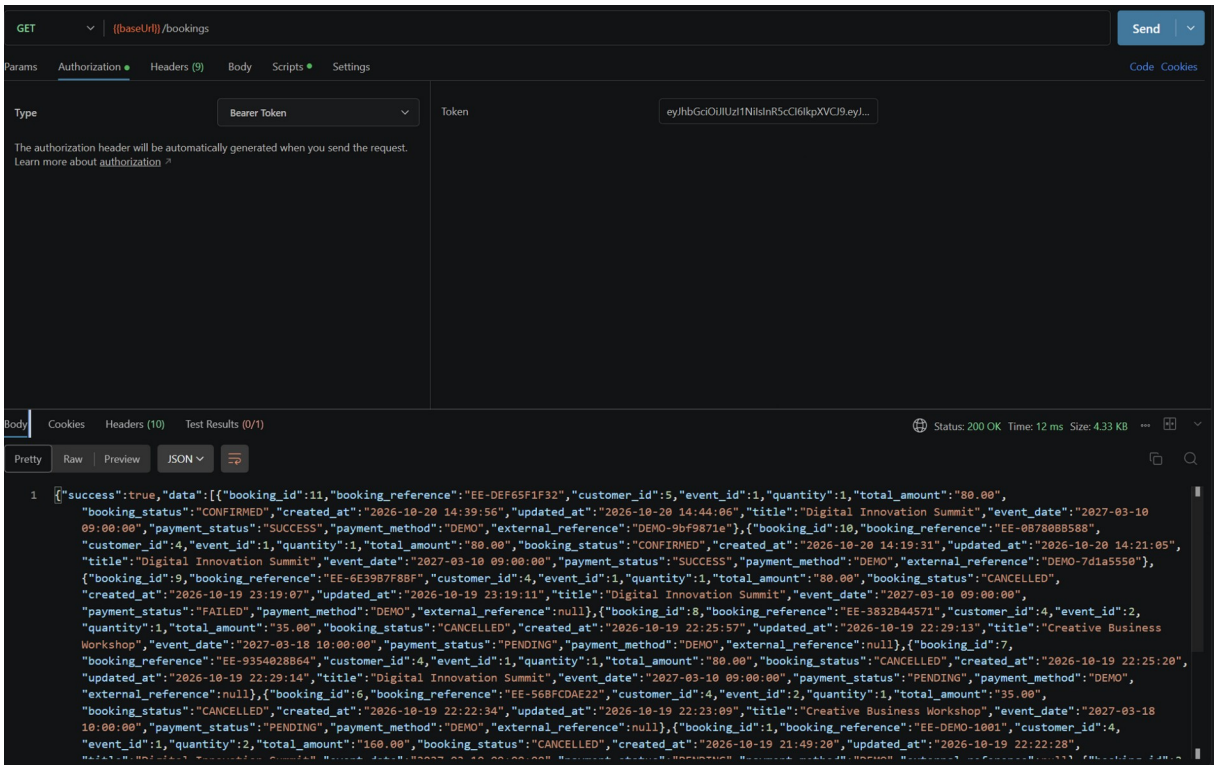
Admin metrics with the valid API key — 200 OK



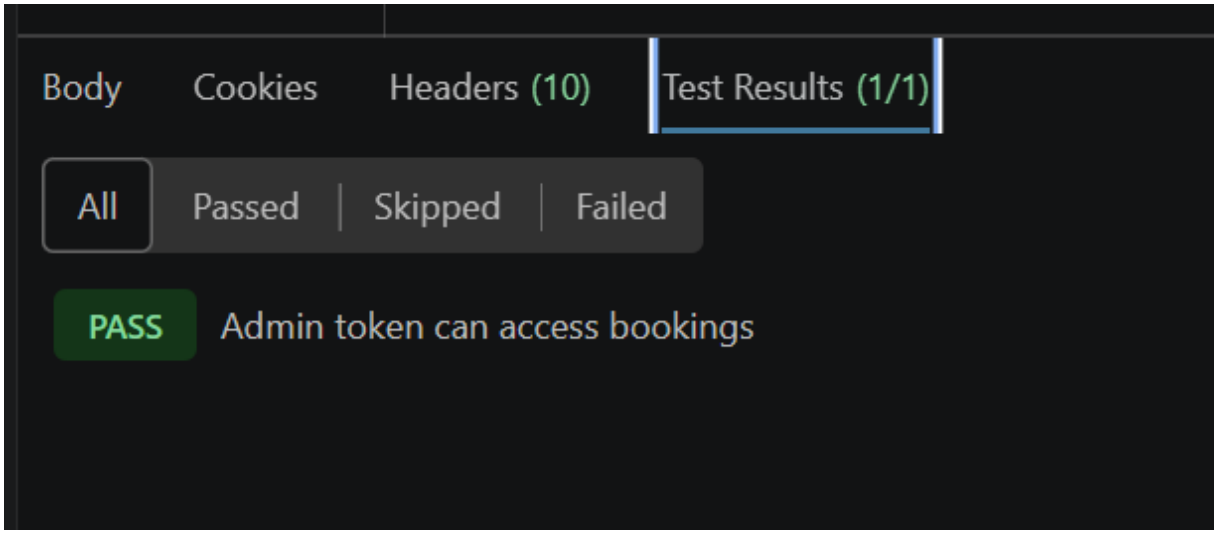
Invalid/expired JWT — 401 Unauthorized



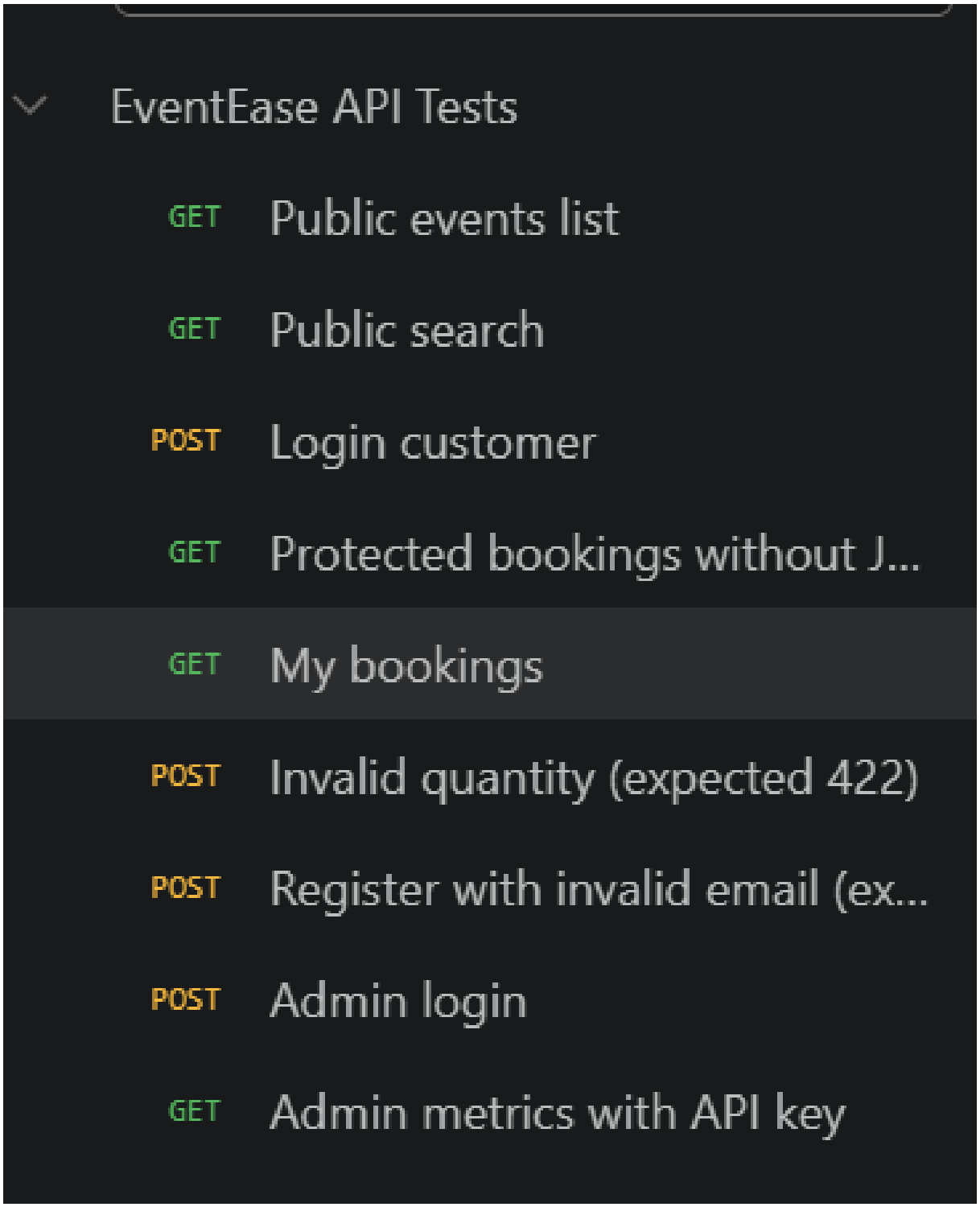
Bookings endpoint using the fresh administrator token — 200 OK



Postman assertion passed: Admin token can access bookings (1/1)



Current EventEase API Tests collection request list



Note: The screenshots above show selected current results. The full collection contains nine requests. Older test evidence remains in Appendix A. Before final submission, replace any remaining personal-detail or reflection placeholders and verify that the SQL export contains the latest sample data.